UNIVERSITY OF WATERLOO

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

ECE-124 LAB MANUAL

V3.0

LAB 2

Course: ECE-124 Digital Circuits and Systems(with Verilog)

1 Lab 2 – Verilog - Combinational Circuits 1– Simple ALU (Dataflow / Structural Verilog)

The primary goal of this lab session is to gain more experience with Verilog for combinational logic design. Some new Verilog components will be introduced along with their associated data format requirements.

1.1 Lab2 Intended Learning Outcomes

By the end of this lab, students should be able to:

1) **IMPLEMENT** Verilog design modules
2) **APPLY** Verilog modules into for multiple instances
3) **UNDERSTAND** basic Verilog functions (Adder, Decoder and Multiplexer) designs
4) **CREATE** a digital design for a simple Arithmetic Logic Unit (ALU) and confirm with simulation and downloading to the LogicalStep board

1.2 Lab2 Outline

Attendance will be taken.

The lab starts with a brief review of the design entry methods used in Lab1.

Verilog modules will be re-used in the system design.

Then the Lab will be getting into background details for the Lab2 project

1 Project Setup for Lab2
2 Lab2 Part A: New Verilog Component – What is a Seven Segment Decoder function?
3 Lab2 Part B: New Verilog Component - What is a Multiplexer or MUX function?
4 Lab2 Part C: Creating a Simple Logic Processor from a Multiplexer Design
5 Lab2 Part D: New Verilog Component – What is an Adder function?
6 Lab2 Part E: A Little bit on Signal Concatenation
7 Lab2-Project Brief

1.3 Lab2 Activities

1.3.1 Recall from Lab1:

In the Lab1, some basic gate functionality was created in the FPGA. Some of the tools and utilities available within the Quartus Prime FPGA development environment were briefly explored. The top-level design was schematic-based. A subordinate block in schematic form was developed and added into the design hierarchy. Later the design went through a process to "synthesize" the design into a logical gates model so that functional simulations could be completed. Later a functional STIMULUS file was created to stimulate the synthesized gate design in simulation. After completion, the simulation results were compared against the truth tables for the gates implemented.

A Verilog design that was functionally equivalent to the schematic version was also created. Then it was added to the top level of the FPGA design.

As a final design step, some output polarity control was added (one in schematic form, one in Verilog form).

Simulations were used to confirm that the design, whether in schematic or Verilog form, were equivalent.

Then the two methods for design entry were used to create another pair of blocks. This second pair of blocks offered the functionality of a Signal Polarity control. The concepts of Active-HI and Active-LO signals were covered. The new enhancements were added to the top-level design and further simulations were accomplished.

Having completed a functional verification of the full Lab1 design entry with simulation, we then modified the design to accommodate small differences that are required for operation on the LogicalStep board. We noted that the push-button inputs (pb[3:0} have an Active-LOW digital signal format. We inserted inverters into the Lab1 design for those push button inputs. These convert the Active-LOW pb inputs to an Active-HIGH format. After this change, a full design COMPILATION was run so that a download file could be created for loading into the LogicalStep board FPGA. Later, it was confirmed that by observing the LED patterns that the Lab1 design worked in actual hardware.

1.3.2 Lab2 Verilog –Styles

For Lab2 there are just two Verilog coding styles being used:

a) Dataflow: where the relation between inputs and outputs are declared using logical equations
b) Structural: where you use previously created Verilog designs and are using them in a different design as component blocks

Lab1 was VERY BASIC in scope, and it used only one style of Verilog coding (DATAFLOW). It just had basic Boolean equations and two-input gates.

Lab2 will use that style as well and it will also use a second Verilog style called STRUCTURAL.

Quite often a Verilog design is constituted from smaller Verilog blocks connected to form a more complex Verilog function. Using a hierarchy of Verilog blocks, the STRUCTURAL Verilog style is implemented. This will be the style required at the top-level of the design for Lab2 and remaining labs. No Dataflow style at the top-level will be allowed.

Before we begin to use Structural Verilog, we first must understand the difference between a Module Declaration and a Module Instantiation.

You already used a Module Declaration in Lab1 for the verilog_gates file. See the example of the syntax below for the file called verilog_gates. For the Module Declaration, the port names and their signal directions must be inserted. Note that the name declared in the Module Declaration must match Verilog filename (when the file is saved in the project directory). In the example below the port name and the port signal directions are evident.

```
1   module verilog_gates
2
3   (
4       // Input Ports
5       input XOR_IN1,XOR_IN2,OR_IN1,OR_IN2,NAND_IN1,NAND_IN2,AND_IN1,AND_IN2,
6
7       // Output Ports
8       output XOR_OUT, OR_OUT, NAND_OUT, AND_OUT
9   );
10
```

The Lab1 Verilog file called verilog_gates.v is shown below:

```
1   module verilog_gates
2
3   (
4       // Input Ports
5       input XOR_IN1,XOR_IN2,OR_IN1,OR_IN2,NAND_IN1,NAND_IN2,AND_IN1,AND_IN2,
6
7       // Output Ports
8       output XOR_OUT, OR_OUT, NAND_OUT, AND_OUT
9   );
10
11  // verilog logic operators
12  // XOR:     ^
13  // OR:      |
14  // NOR:     ~(.. | ..)
15  // AND      &
16  // NAND     ~(.. & ..)
17  // XNOR     ~(.. ^ ..)
18
19  // fill in the missing operators below:
20
21      assign AND_OUT = AND_IN1 & AND_IN2;
22      assign NAND_OUT= ~(NAND_IN1 & NAND_IN2);
23      assign OR_OUT  = OR_IN1 |  OR_IN2;
24      assign XOR_OUT = XOR_IN1 ^ XOR_IN2;
25
26  endmodule
27
```

After a Module is declared, and its Verilog file is saved, the inclusion of the module into a higher-level hierarchy can be made in a Verilog design. To do that, a companion Module Instantiation statement is inserted into the higher-level file.

For our Lab1 example, the LogicalStep_Lab1_top file is shown below for when the Lab1 project did NOT have the Polarity Control blocks or the inverters for the push-0button inputs. A Module Instantiation statement is inserted for each copy of the verilog_gates module being used.

Below, the Verilog file for verilog_gates.v is shown. One instantiation (U0) is shown with ports and their port connections separated by commas. The second instantiation (U1) has each port on a different line. For each port in the Verilog_gates module instantiation, a "." Is placed before the port name and the connection to a net in the higher-level LogicalStep_Lab1_top.v file is included in brackets.

```
module LogicalStep_Lab1_top (
    input    sw1,
    input    pb1, pb0,
    output   leds3,leds2,leds1,leds0,
    output   leds7,leds6,leds5,leds4
);

// verilog_gates module instantiations

verilog_gates u0 (
.XOR_IN1 (pb0), .XOR_IN2 (pb1), .OR_IN1 (pb0), .OR_IN2 (pb1), .NAND_IN1 (pb0), .NAND_IN2 (pb1), .AND_IN1 (pb0), .AND_IN2 (pb1),
.XOR_OUT (leds3), .OR_OUT (leds2), .NAND_OUT (leds1), .AND_OUT (leds0)
);

verilog_gates u1 (
.XOR_IN1 (pb0),
.XOR_IN2 (pb1),
.OR_IN1 (pb0),
.OR_IN2 (pb1),
.NAND_IN1 (pb0),
.NAND_IN2 (pb1),
.AND_IN1 (pb0),
.AND_IN2 (pb1),
.XOR_OUT (leds7),
.OR_OUT (leds6),
.NAND_OUT (leds5),
.AND_OUT (leds4)
);

endmodule
```

NOTE the Module Instance, the text of each of port names must match exactly with the text of those ports defined in the Module Declaration of the Verilog file being used. The port name text is case-sensitive.

This example above can be used as a reference for the module instantiation exercises in this lab.

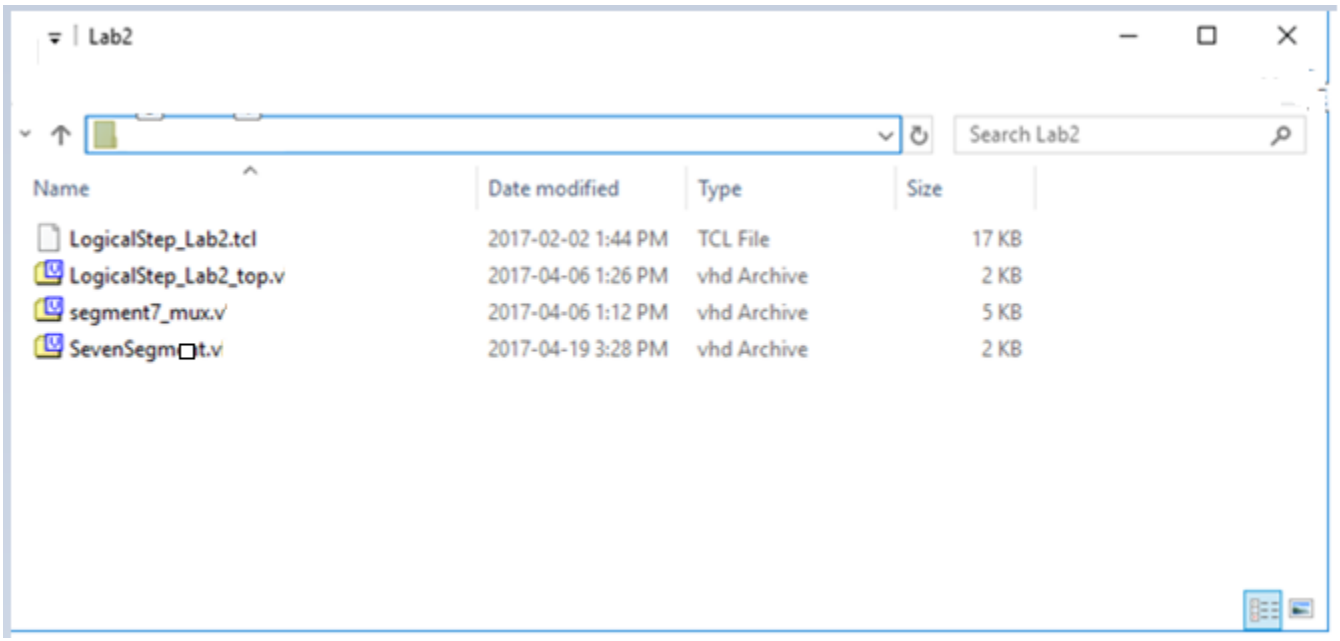
Some syntax generalities to notice from the above example:

1) The **port names** of module instances in the Lab1 Verilog example must match **exactly** as the Module Declaration port names.
2) The last element in the list of ports etc. must not have a terminating comma.
3) All module names and net names or port names are case-sensitive in Verilog.

4) The port order if the Module Instance can be different from the Module Declaration port order when net connections are included in brackets.

1.3.3 Project Setup for Lab2

Using the Windows File Explorer, go into your C:drive/user/<name>/ECE-124 folder directory.
Go to LEARN and download the Lab2 Zipped folder "Lab2.zip". Copy it into the ECE-124 folder. Extract the contents into the ECE-124 project folder. This will create the Lab2 project folder and some starter files.



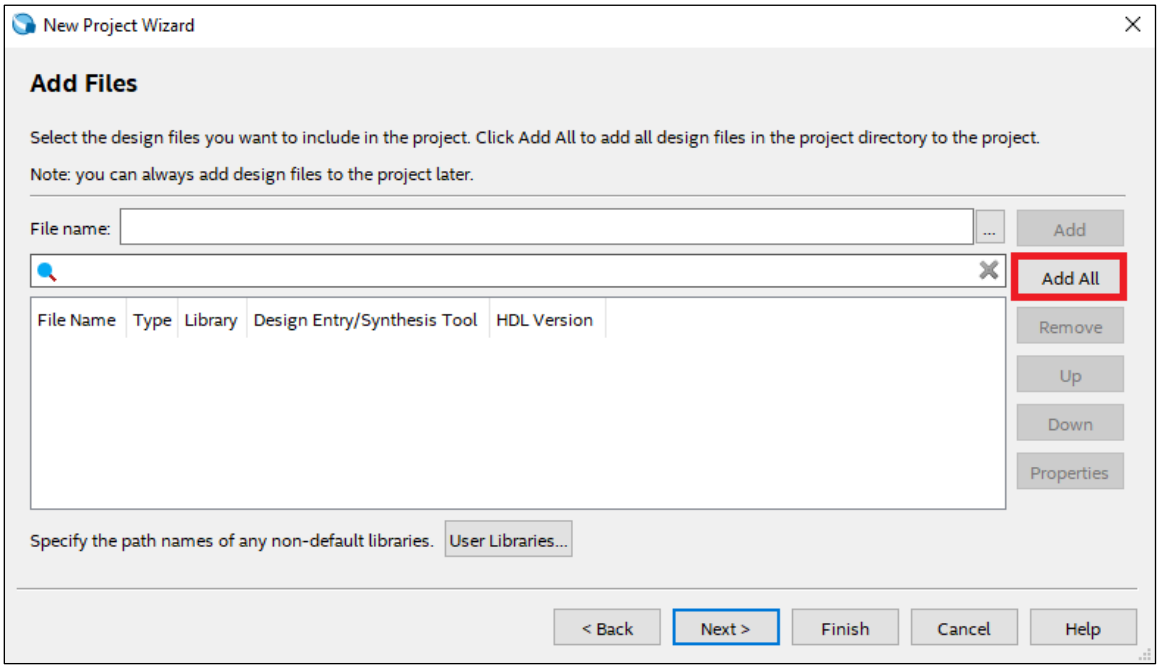
Start up the Quartus Prime platform and begin a new project (Using FILE>New Project Wizard).

Click NEXT to go to the next slide.
The project parameters will now be entered (**MAKE SURE THAT THERE ARE NO SPACES USED**).

Project Folder: **C:/users/<name>/ECE-124/Lab2**
Project Name: **LogicalStep_Lab2**
Project Top Level: **LogicalStep_Lab2_top**

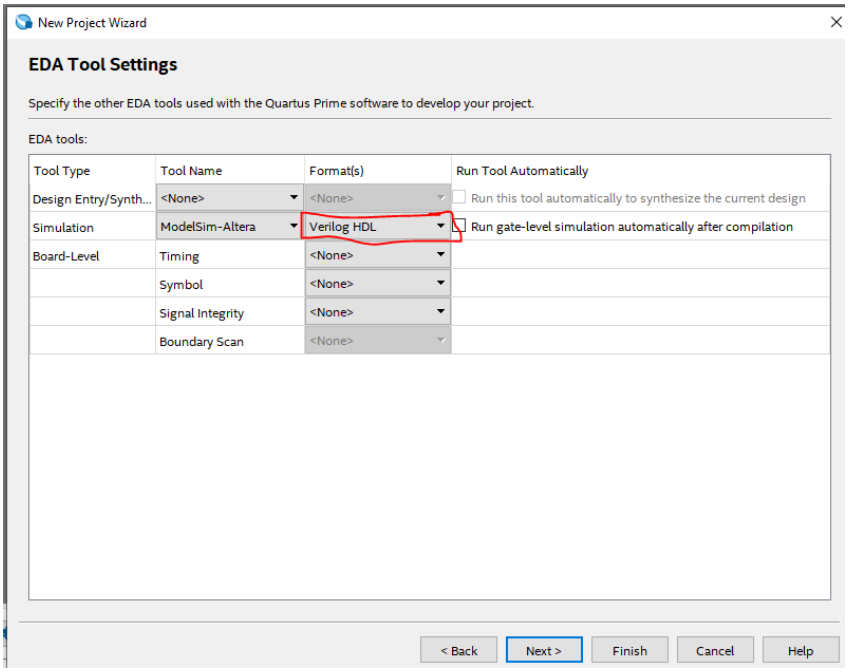
Click NEXT to go to the next slide.

Then click NEXT on the New Project Wizard Dialog Window. Then click NEXT again (with Empty Project). Then the dialog box below shows up.



Click on the ADD All button. This will bring the starter design files (downloaded from Learn) into the Quartus LogicalStep_Lab2 project

.



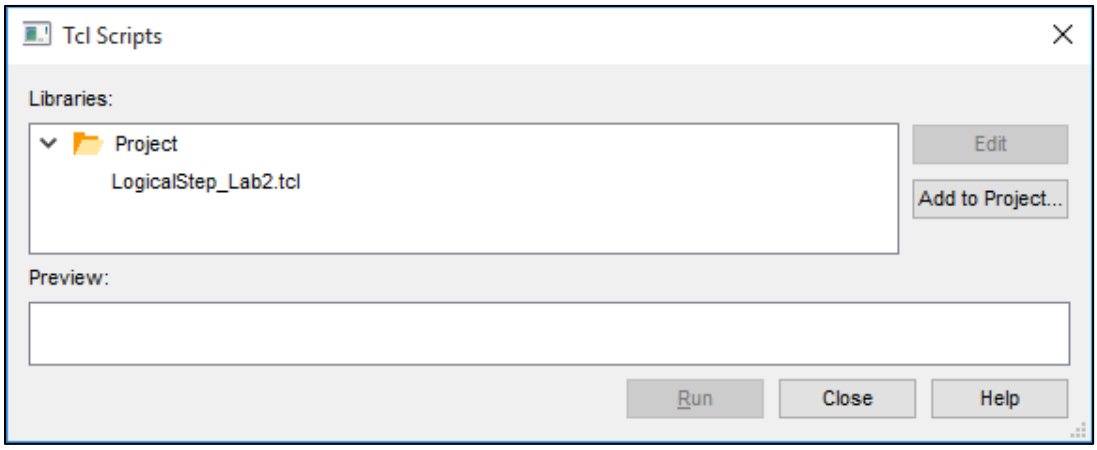
Click NEXT on dialog windows for Family, Device & Board Settings until you reach the following window (EDA Tool Settings**)**
Then choose (1) **Modelsim-Altera** and (2) **Verilog** from the drop-down menu according to the following figure:

Now the Project setup is entered. Click FINISH. DO NOT OPEN ANY STARTER FILES AT THIS POINT.

IMPORTANT: Do this step next.

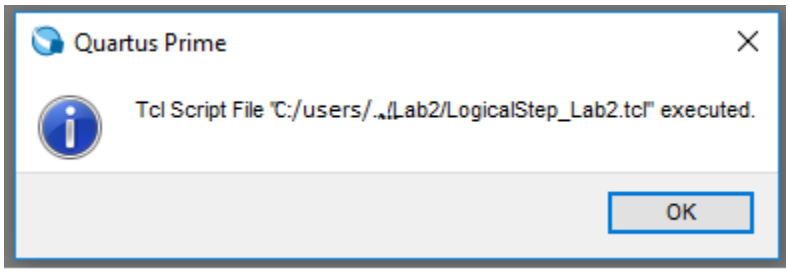
Next, in Quartus, the Lab2 TCL script must be run to assign the FPGA device type, the FPGA pin assignments for the FPGA that are reserved for the LogicalStep FPGA and finally the project LogicalStep_Lab2 is created and opened.

Go to the Tools tab and SELECT the Tcl Scripts option. The dialog box below should appear:



SELECT the TCL file "LogicalStep_Lab2.tcl" and then click on the RUN button as in the figure above.

AFTER ABOUT 5 seconds, the following should appear when it is finished.



IF IT TAKES LESS THAN 5 SECONDS, PLEASE INFORM THE LAB INSTRUCTOR. Things may not have been processed properly.

Click the OK Button and close the TCL Scripts window.

The starting version for the LogicalStep_Lab2_top.v file is shown below:

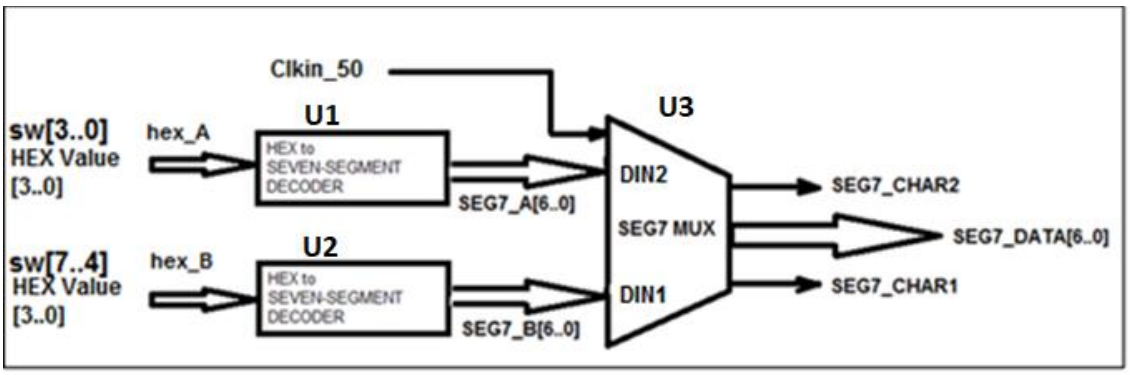
```
1   module LogicalStep_Lab2_top
2   (
3      input          rst_n,      //reset in
4      input          clkin_50,   //clock in
5      input    [7:0] sw,
6      input    [3:0] pb_n,
7
8      output   [7:0] leds,
9      output   [6:0] seg7_data,
10     output         seg7_char1,
11     output         seg7_char2
12  );
13
14  //these are used as intermediate signals
15     wire  [3:0] hex_A, hex_B;
16     wire  [6:0] seg7_A, seg7_B;
17
18
19  // wire assignments
20     assign hex_A = sw[3:0];
21     assign hex_B = sw[7:4];
22
23
24  //module instantiations are here
25     SevenSegment u1 (
26        .hex      (hex_A),
27        .sevenseg (seg7_A)
28     );
29
30     SevenSegment u2 (
31        .hex      (hex_B),
32        .sevenseg (seg7_B)
33     );
34
35     segment7_mux u3 (
36        .clk  (clkin_50),
37        .din2 (seg7_A),
38        .din1 (seg7_B),
39        .dout (seg7_data),
40        .dig2 (seg7_char2),
41        .dig1 (seg7_char1)
42     );
43
44
45
46  endmodule
```

The starer version has some things already declared for you. The inputs and outputs are defined for the LogicalStep_Lab2_top design.

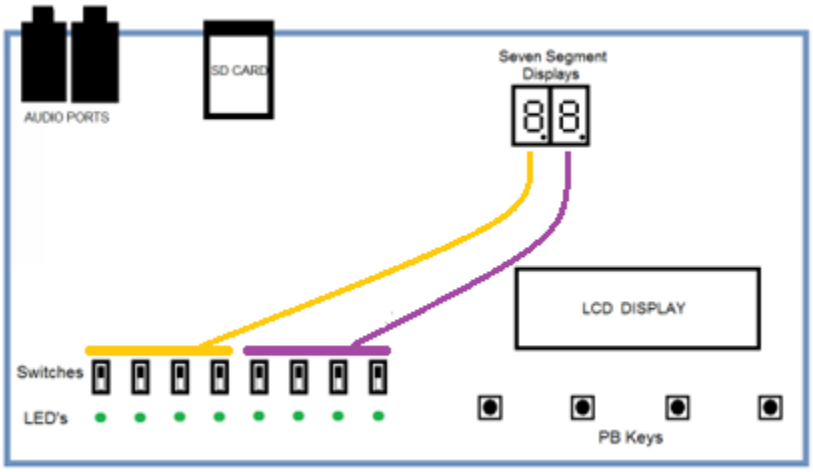
1) The push-button input port names have been corrected to denote that they use ACTIVE-LOW signal format (_n suffix). This is different from what was used in Lab1.
2) Some nets are multi-bit. The dimension of the multi-bit nets are given as [msb:lsb]. So pb_n has 4 input bits, sw has 8 input bits, and the leds have 8 output bits. Other ports will be used later in the lab.
3) After the Module Declaration section, some internal nets (wires) are defined and are multi-bit as well. hex_A has 4 bits, hex_B has 4 bits, seg7_A and seg7_B each have 7 bits.
4) There are two connections assigned. hex_A connects to sw[3:0] and hex_B connects to sw[7:4].
5) Three Module Instantiations are made with some of their connections to intermediate (or internal) signals and some to output ports of the LogiocalStep_Lab2_top design..

Provided Module Instantiations for Lab2:

1) Sevensegment.v is provided to you for generation of the seven-segment patterns that correspond to 4-bit hex values arriving at its input. These are the U1 and U2 instances.
2) Segment7_mux.v is provided to drive the external dual-seven-segment displays on the LogicalStep board. It takes two 7-bit inputs from the seven-segment decoders discussed in the next section. This is the U3 instance.

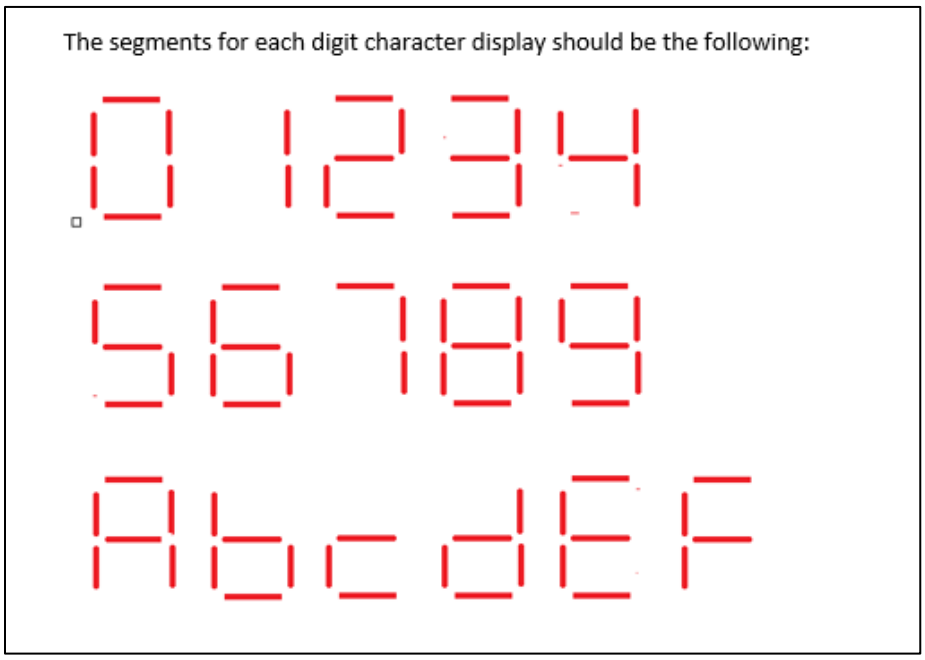


Next, run a FULL compilation process. This can be done by going to the Processing TAB and then selecting "Processing>Start Compilation" option. This will create a download file for testing on the LogicalStep board. Setting the switches for the DIGIT1 and DIGIT2 displays on the LogicalStep board will cause seven-segment patterns to be shown.

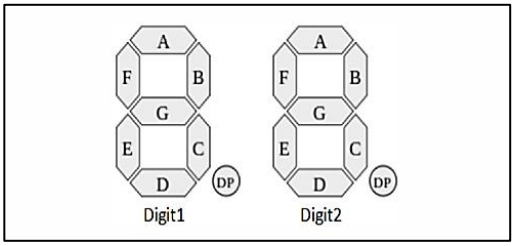


BUT, the patterns will only be initially available for 4-bit hex values of '0' and '1' for those switch groups (switches 7:4 control the Digita1 pattern and switches 3:0 control the Digit2 pattern.

You will need to modify the sevensegment.v file for the bit patterns for the other 4-bit hex values so that you have seven-segment patterns for **all hex values between 0 and F**.



1.4 Lab 2 Part A: NEW Verilog Component - What is a Seven Segment Decoder?



The LogicalStep board has two seven segment displays available.

To drive each display, we typically use a “hex to seven-segment decoder”. Hex input values (4 bits) are used to represent hex values, and the decoder converts the 4-bit hex value to a pattern of 7 bits to drive the seven LED segments of a display digit. A snip of a Verilog example of this function is shown below with some seven-segment codes missing and need to be filled in by you. A Verilog Conditional operator (or ternary operator) statement is used

```
module SevenSegment (

   input [3:0] hex,         // The 4 bit data to be displayed

   output[6:0] sevenseg   // 7-bit outputs to a 7-segment
);


// The following statements convert a 4-bit input, called dataIn to a pattern of 7 bits
// Each segment turns on when it is '1' otherwise '0'
// here is the seven segment display map:
//
//         +---- a -----+
//         |            |
//         |            |
//         f            b
//         |            |
//         |            |
//         +---- g -----+
//         |            |
//         |            |
//         e            c
//         |            |
//         |            |
//         +---- d -----+

//                                hex bits      sevenseg
//                                    3210       GFEDCBA
      assign sevenseg = (hex == 4'b0000) ? 7'b0111111 :
                        (hex == 4'b0001) ? 7'b0000110 :
                        (hex == 4'b0010) ? 7'b[redacted] :
                        (hex == 4'b0011) ? 7'b[redacted] :
                        (hex == 4'b0100) ? 7'b[redacted] :
                        (hex == 4'b0101) ? 7'b[redacted] :
                        (hex == 4'b0110) ? 7'b[redacted] :
                        (hex == 4'b0111) ? 7'b[redacted] :
                        (hex == 4'b1000) ? 7'b[redacted] :
                        (hex == 4'b1001) ? 7'b[redacted] :
                        (hex == 4'b1010) ? 7'b[redacted] :
                        (hex == 4'b1011) ? 7'b[redacted] :
                        (hex == 4'b1100) ? 7'b[redacted] :
                        (hex == 4'b1101) ? 7'b[redacted] :
                        (hex == 4'b1110) ? 7'b[redacted] :
                        (hex == 4'b1111) ? 7'b[redacted] :
                                           7'b0000000 ; //for other values

endmodule
```

Your task is to use the seven-segment map to set the appropriate digital 1's or 0's to drive the seven segments (A through G) for all hex digit values.

For the hex values of 0 and 1, the seven segment values are shown.

Question: For this example, how big of a task do you feel it would be to enter the function above with just schematic gates??

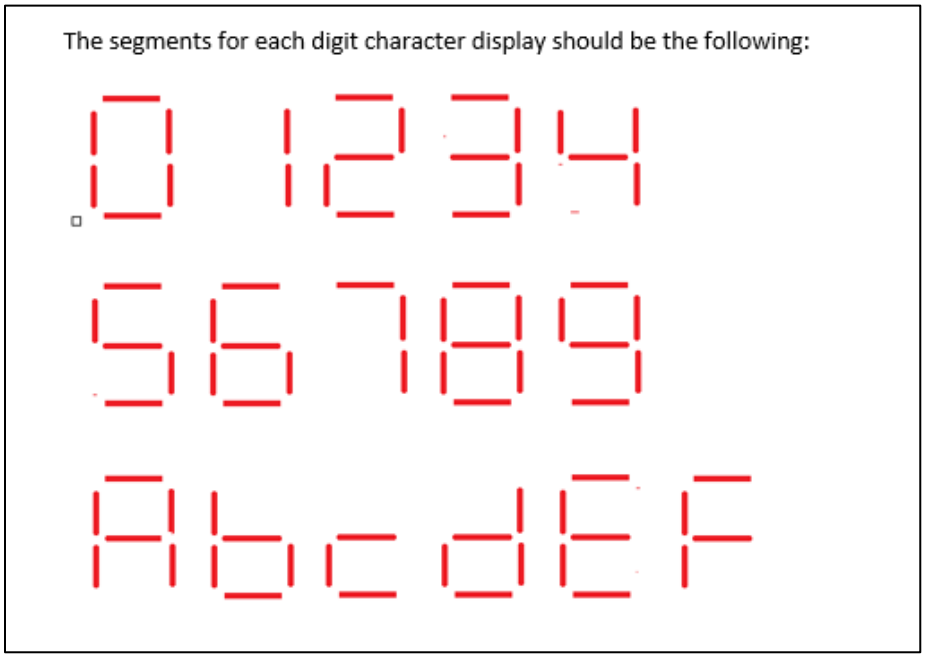
TAKE-AWAY:

➔Any Hardware Description Language (HDL) method is often much more efficient than the schematic design entry method.

The Verilog Conditional Operator is of the form:

(condition) ? choice if condition is TRUE : choice if condition is FALSE

In the arrangement of the conditional operator in the sevensegment.v file, the operators are configured in a sequential fashion. A first condition is used to text the hex value input of “0”. If the condition for the first comparison is TRUE, then the seven-segment pattern for a ‘0’ is output from the decoder. If the first condition is FALSE, then the next condition is tested (for a “1” etc. and on and on. If none of the conditions are TRUE, then no segments are illuminated.

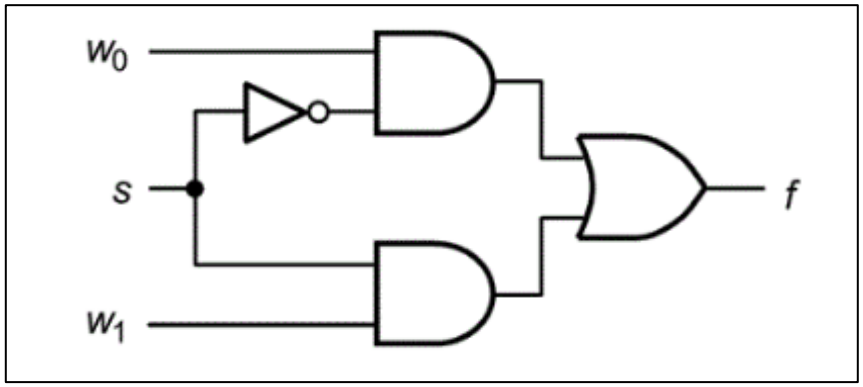


You can make your changes in the sevensegment.v file, save it and then do a full recompile of the LogicalStep_Lab2 project and then download it to see the updated seven segment patterns for the other hex values.

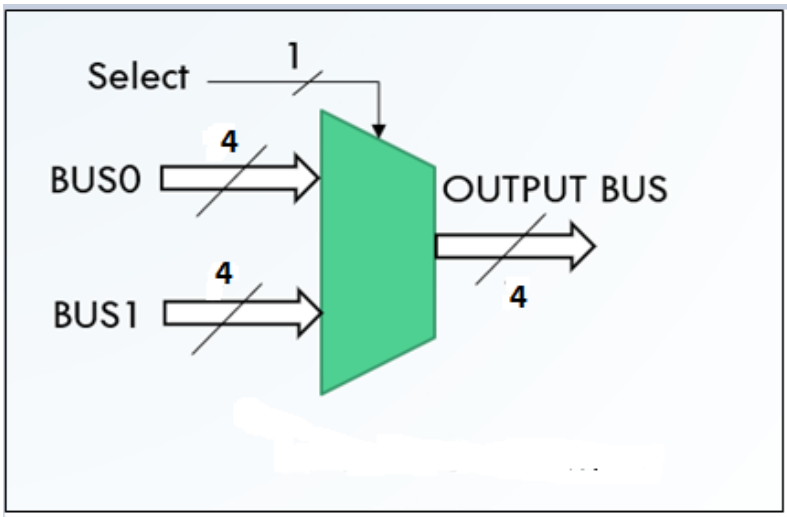
1.5 Lab 2 Part B: NEW Verilog Component - What is a Multiplexer or MUX function?

Multiplexers are used to select different data sources of input to a downstream function input or process. The selection is controlled by the state of the SELECT control inputs (see Figure 63).

Multiplexers can be found in a number of input/output ratios (e.g.: 2 to 1, 4 to 1, 8 to 1 …)



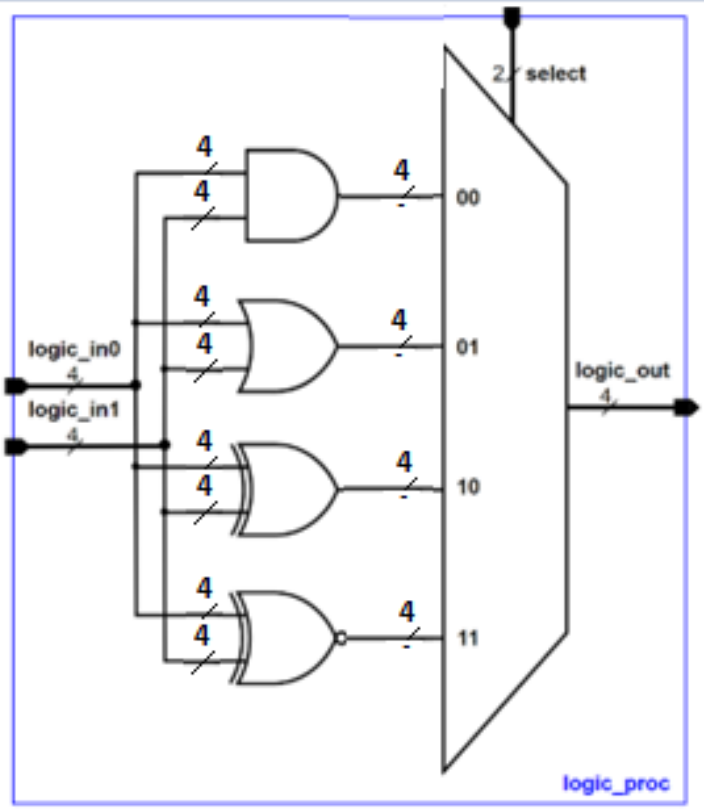
The figure on the left is a simple 2 to 1 multiplexer or MUX function. Its output function "f" will pass thru the w0 input value when the "select" control input "s" is in a LOW state (or a "0"). When "s" is HIGH (or "1") the output function "f" will equal the value from the w1 input.



A graphical representation example of a 4-bit 2 to 1 multiplexer is shown here for your reference on the left. A Verilog companion is shown below. All busses are 4 bits wide. The 1-bit selector can direct either of the 2 input ports to the output port.
Create this mux_4bit_2_to_1.v file shown below in your Lab2 project folder. This will be needed later. It does not need to be added as an instance to the LogicalSTep_Lab2_top.v file yet.

```
1  module mux_4bit_2_to_1 (
2      input  [3:0] din_A, din_B,
3      input        selector,
4      output [3:0] dout
5      );
6
7      assign   dout = (selector == 1'b0) ? din_A : din_B;
8
9  endmodule
10
11
```

1.6 Lab2 Part C: Creating a Simple Logic Processor from a Multiplexer Design



Using your knowledge of designing a Verilog 4-bit, 2 to 1 MUX from Part B, and your observations with the daisy-chained conditional operator construct used in the seven segment decoder to create a simple Logic Processor with four logic processing choices. See the diagram at the left.

There will be just TWO hex input values and a two-bit port for a selecting any one of four logic operations. Then inside the Logic Processor, there will be four different selections available (AND, OR, XOR, XNOR). Note the value of the select port for each logic choice in the diagram.

You can use the bitwise operators on the multi-bit hex values (e.g. result= hex_A[[3:0] & hex_B[3:0].

Make sure that you save this new Logic Processor block with the same name as what you use in the Module Declarat6ion.

1.6.1 Experimenting with the Logic Processor Design and Adding Push Buttons as Selectors

Before adding the Logic_Processor function to your LogicalStep_Lab2_top file there will be another function to be created first. This Verilog component will be a function that inverts the pb_n inputs. The outputs from this function will be the pb's used to select the logic function choice in your Logic_Processor function.

```
1   module pb_inverters (
2       input  [3:0] pbin,
3       output [3:0] pbout
4       );
5
6       assign   pbout = ~(pbin);
7
8   endmodule
9
10
```

Create a new Verilog block called pb_Inverters.v with the example code at the left. After saving this block, insert an Instance of it in the LogicalStep_Lab2_top level.

Connect the four pb_n input pins (from the top-level) to the inputs of the instance of the pb_Inverters block. You will have to add a new signal group called pb[3:0] and connect it to the pb_Inverters output port at the tp-level.

Now, for the Logic Processor module, add an instance for it into the LogicalStep_Lab2_top. Connect the hex_B and hex_A signal groups, previously used, to the 2 input ports (logic_in1, logic_in0 resp.) of the

Logic Processor instance. Connect the Logic_Processor instance select port (2 bits) to pb[1:0] (which come from the outputs of the pb_inverters instance).

Connect the Logic Processor instance outputs to leds[3:0].

Do an **Analysis and Synthesis** compile. Correct any compile errors. to create a gate-level model for simulation.

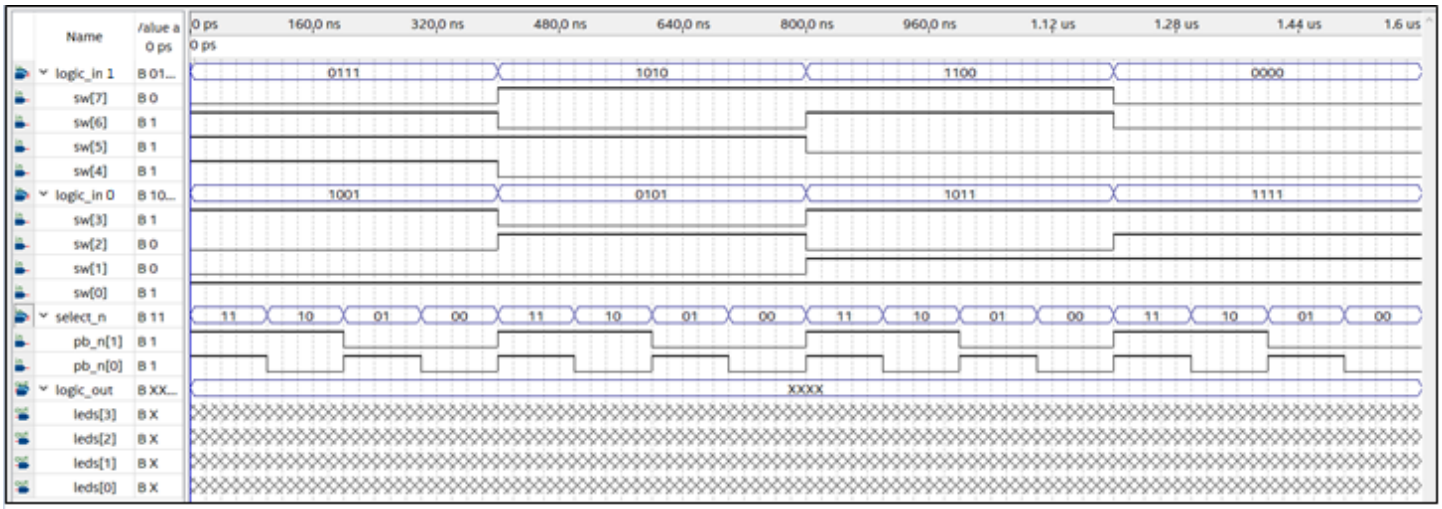
1.6.2 SIMULATING THE LOGIC PROCESSOR

Then, start up a new simulation window to simulate this function with the following stimulus. In the simulation window **add the nodes for all pb_n, sw inputs and leds outputs**. Under the EDIT TAB, set the End Time to 1.6 usec. Then select all of sw[7:4] nodes, right-click and group them into a group called **logic_in1.** Then select each of **sw[3:0]** nodes], right-click and group them into a group called **logic_in0.**

Then select each of pb_n[1:0] nodes, right-click and group them into a group called **select_n**. Then select each of **leds[3:0] nodes** and group name them as **logic_out**.

Note that each of the pb_n input pins MUST BE represented as ACTIVE LOW. The select_n group will be set to change every 100 nsec.

The logic_in0 and logic_in1 values will be set to change every 400 nsec. Thus, for each pair of logic_in1, logic_in0 there should be four logic operations occurring based on the selector_n value. When running the simulation, you should see the logic_out outputs change to match the logic operation being done on the inputs. The simulation stimulus for each Team is to be set up like the following:



HINT: Use the COUNT Icon in the menubar of the simulator to create the counting intervals for the above groups. For the logic_in1, logic_in0 you can manually select each interval and then edit them (double-clicking) to match the above. Ask the LI or TA for assistance if needed.

Keep the resulting Logic Processor simulation for a brief presentation during your Lab2 DEMO

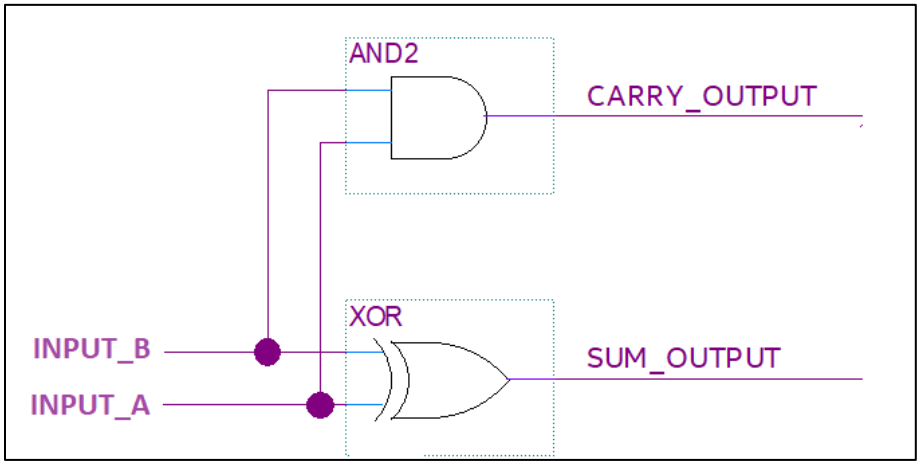
1.7 Lab2 Part D: NEW Verilog Component - What is an Adder function?

An Adder can be of very useful in digital logic designs. It can vary in size and in performance.

To begin, we will discuss a typical adder design just having two single-bit logic inputs. The Adder will generate two outputs based on input logic levels. See the following truth table:

INPUT_B	INPUT_A	SUM OUTPUT	CARRY OUTPUT
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

This functionality can be implemented with logic gates in the following arrangement:



HALF ADDER (single bit)

Now this is all well and good for adding just two single-bit operands. But what if we want to build an Adder that must add inputs having more than just one bit? Inputs that represent values greater than just a 1 or a 0 (ie: for a single-bit operand) will have to use more than one bit for each input quantity.

In such situations, there are multiple bits per input value. How do we systematically process multiple bits for an input? And if there is a "Carry_Output" from a lower order bit adder, how do we include it into the higher order single-bit adders? We need to add a modest amount of complexity to the above single-bit logic adder design.

Let's expand the above truth table that we just used.

First, the name of the column SUM_OUPUT to HALF ADDER SUM_OUTPUT and the Carry_Output to HALF ADDER CARRY_OUTPUT, respectively.

The yellow highlighted part of the table below is identical to the previous one except that it is repeated to cover the span of the new input of CARRY_IN values.

Now the CARRY_IN is inserted into non-highlighted part of the table.

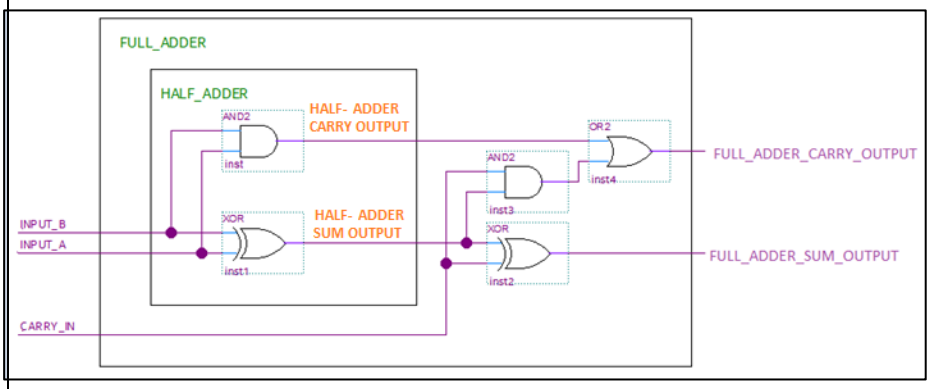
INPUTS TO HALF ADDER		INPUTS TO FULL ADDER			OUTPUTS	
INPUT _B	INPUT _A	HALF ADDER SUM OUTPUT	HALF ADDER CARRY OUTPUT	CARRY_IN (from lower adder carry-out)	FULL ADDER SUM OUTPUT	FULL ADDER CARRY OUTPUT
0	0	0	0	0	0	0
0	1	1	0		1	0
1	0	1	0		1	0
1	1	0	1		0	1
0	0	0	0	1	1	0
0	1	1	0		0	1
1	0	1	0		0	1
1	1	0	1		1	1

The CARRY_IN input (along with the two outputs of the HAFL-ADDER) is now included to generate a FULL ADDER function.

Specifically, when CARRY_IN is 0, the FULL_ADDER_SUM_OUTPUT is the same as the HALF_ADDER_SUM_OUTPUT and the FULL_ADDER_CARRY_OUTPUT is the same as the HALF_ADDER_CARRY_OUTPUT.

But when the CARRY_IN is a 1, the FULL_ADDER outputs must change to generate the proper FULL_ADDER_SUM_OUTPUT and FULL_ADDER_CARRY_OUTPUT signals.

The above truth table can be represented by the circuit below. The original gates in the highlighted part of the above table, are what is described as a “HALF-ADDER”. When the new gates are added to handle the CARRY_IN input, the whole circuit is described as a FULL_ADDER.



FULL ADDER (single bit)

The full adder 1bit circuit is described above in schematic form. Let's call it a full_adder_1bit file.

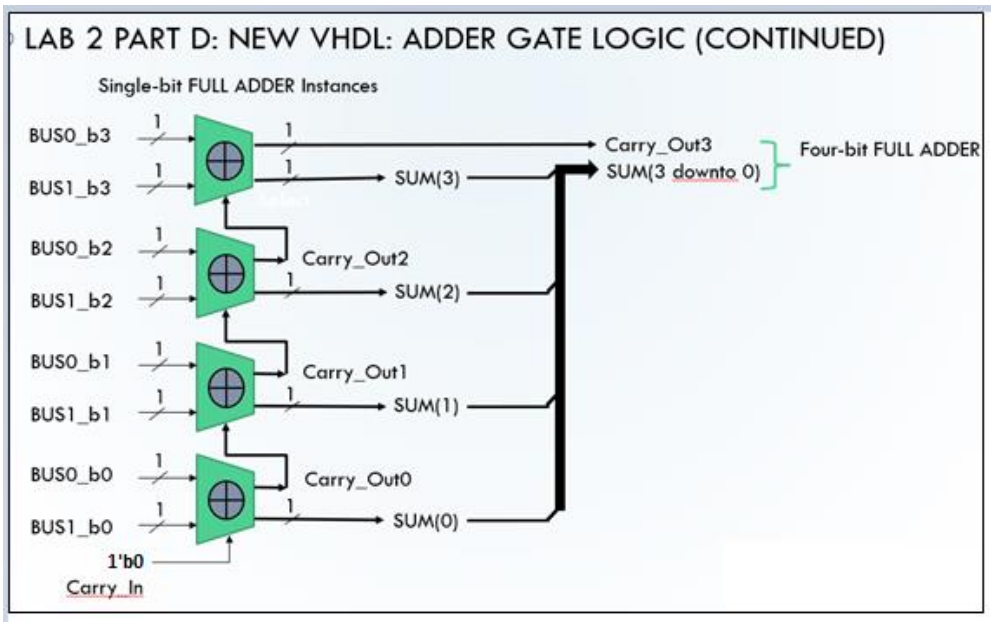
An example of a full_adder_1bit in Verilog is shown below:

```
1
2
3   module full_adder_1bit (
4       input input_A, input_B,
5       input carry_in ,
6       output carry_out, sum_out
7       );
8
9
10  wire  half_sum_out, half_carry_out;
11
12
13  assign half_sum_out   = input_A ^ input_B;
14  assign half_carry_out = input_A & input_B;
15
16  assign carry_out = half_carry_out | (half_sum_out & carry_in);
17  assign sum_out   = half_sum_out ^ carry_in;
18
19  endmodule
20
```

To create larger, multi-bit adders you will have to "daisy-chain" the carry-out of multiple single-bit Full_Adder_1bit instances to the higher order carry-in of the next single-bit Full_Adder. For example, a four bit FULL_ADDER will require FOUR of the above designs, daisy-chained together.

Create Verilog a full_adder_4bit.v Verilog file in the Lab2 project folder.

Then in the full_adder_4bit.v file, add four module instances of the full_adder_1bit. Then connect signals to the instances as shown below.

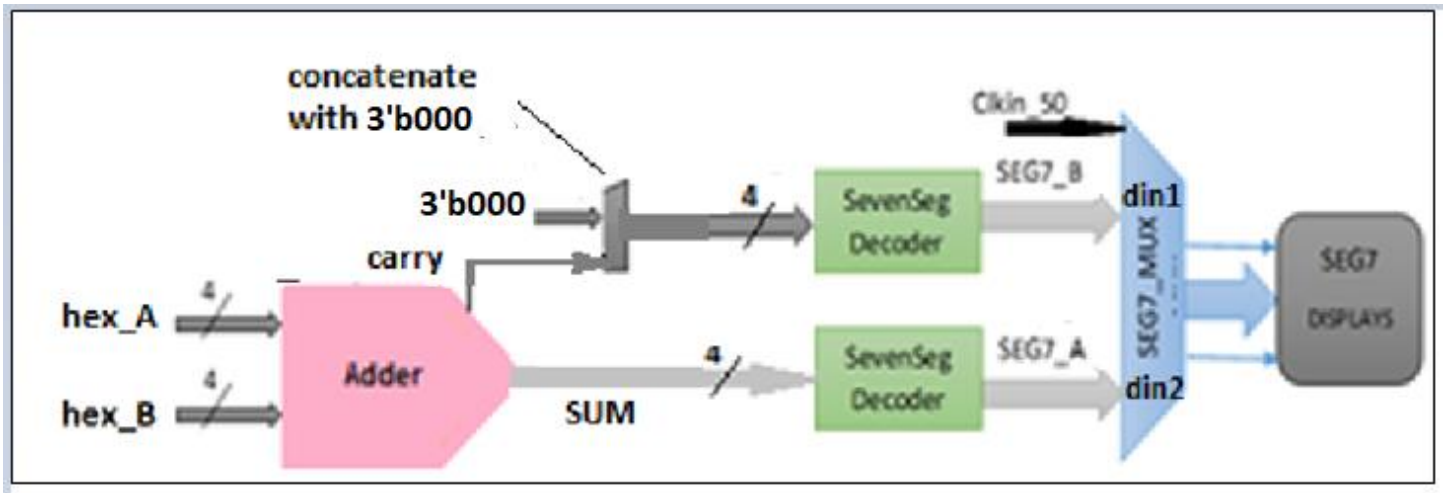


As mentioned above, use four daisy-chained instances of your full_adder_1bit to implement the full_adder_4bit (Verilog Structural design style). Also, assign a **1'b0** (use single quote) to the carry_in port of the full_adder_4bit.

1.7.1.1 Experimenting with the Full_Adder_4bit Design

To test your full_adder_4bit.v design at the LogicalStep_Lab2_top.v level we want to use the Display digits on the LogicalStep board.

Disconnect the hex_A and hex_B signals from the two sevensegment instances from the starter file version of LogicalStep_Lab2_top.



Connect hex_A and hex_B to the 4-bit inputs of the instance of the full_adder_4bit. Also, assign a 1'b0 (use single quote) to the cin port of the full_adder_4bit.

Connect the full_adder_4bit hex_sum output to the sevensegment instance that drives the SEG7_A signal.

The connection for the Carry_Out signal from the full_adder_4bit is explained in the next section (Part E).

1.7.1.2 Lab2 Part E: A Little Bit of Signal Concatenation Verilog

Next, a concatenation of 3'b000 and the full_adder_4bit carry_out signal is required to the sevensegment instance that drives the SEG7_B. Follow the example below to accomplish this.

In order to group signals to other signal groups, you can use the **concatenation operator** in Verilog. The Verilog concatenate operator is ampersand { , }. The two elements are separated by a comma within the braces.

For example, assuming that signal_A is a 3-bit signal and signal_B is a 1-bit signal you can group these two and create a new 4-bit signal named signal_C.

```
--Declare a new 4-bit signal
wire [3:0] signal_C;

--Concatenation
assign_C = {signal_A , signal_B};
```

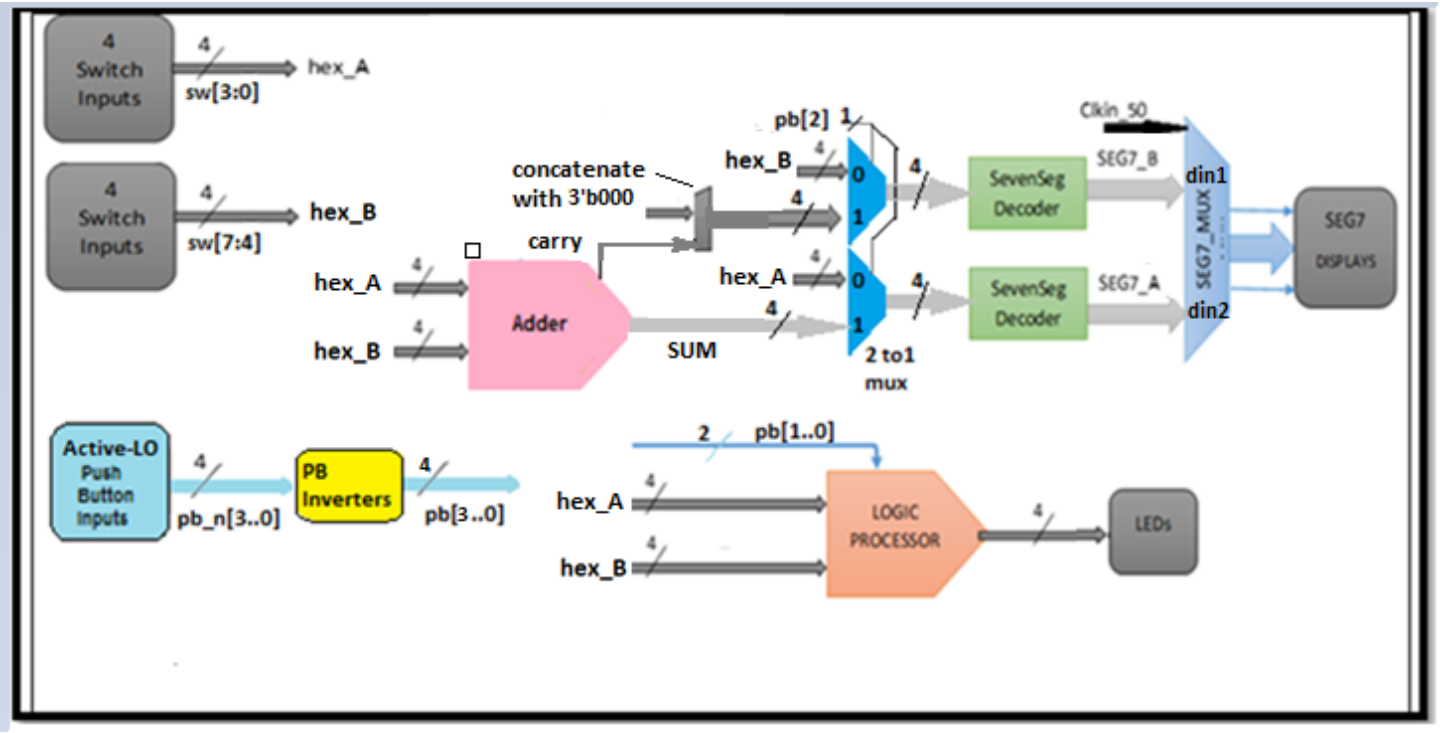
In the concatenation operator example shown above, signal_A will be the MSB of the signal_C group after combining signals. You can use this operator to group onto a 4-bit signal group.

NOTE: The most significant Carry_Out signal for the full_adder_4bit can be considered as the MOST SIGNIFICANT BIT of the SUM (ie: a Bit 5 in this example).

Do a FULL Compilation of the design and download it to the LogicalStep board. Test the 4 bit adder design by setting the hex values for hex_A and hex_B with the switches. The hex formatted sum should appear on Digits 1 and 2. Make sure the the sum digits appear in the correct order on the dual 7-seg display.

1.7.2 Lab2 Project Brief for Demonstration

The previous parts of Lab2 will be used to develop a simple ALU with the LogicalStep_Lab2_top design. An ALU is an Arithmetic Logic Unit, which is a fundamental part of the computer. The schematic diagram of the top level of the lab2 project is shown below.



NOTES:

Verilog STRUCTURAL STYLE MUST BE USED AT THE TOP LEVEL (LogicalStep_Lab2_top.v). NO DATAFLOW CONSTRUCTS WILL BE ALLOWED (logic keywords AND, OR, XOR, NAND, INV etc. or with/select constructs etc.) AT THE TOP LEVEL. But, Concatenation may be used at the top-level.

The pb_inverters, various multiplexers, logic_processor, and full_adder_4bit Verilog files have been developed in the previous part of this lab.

The Logic Processor will process the two operands in a bit-wise fashion. The Logic Processor will offer 4 different logic operations on the operands and the logical results will appear on leds[3:0].

The pb_n[1:0] will be used to select the logic function type according to the following table:

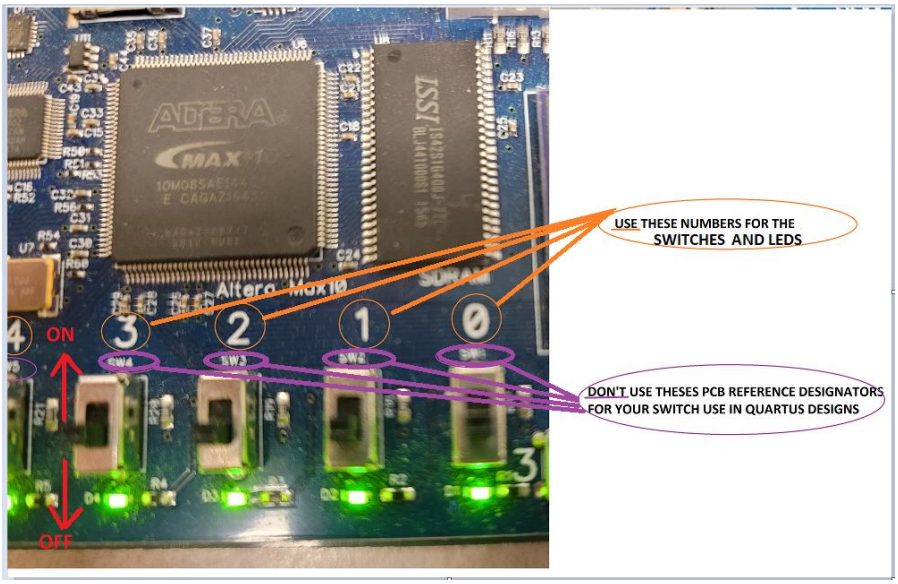
pb_n(1)	pb_n(0)	Logic Function
1	1	AND
1	0	OR
0	1	XOR
0	0	XNOR

The 4 bit Full Adder and hex operands are connected to a small multiplexer network.

When pb([2] is 0, (so pb_n[2] is therefore to be a 1'b1), the multiplexers direct the two hex operand values to the digit displays (hex_A on DIGIT2 and hex_B on DIGIT1).

When pb[2] is 1, (pb_n[2] is therefore to be a 1'b0), the multiplexers directs the Adder value to the digit displays (SUM on DIGIT2 and the concatenation of {3'b000, carry} on DIGIT1).

Do a FULL Compile on the design, resolve any errors, download to the LogicalStep board to test it out. Have the Lab2 Project ready for demonstration during the next Lab Session.



NEXT LAB SESSION: You will be designing a Magnitude Comparator (from scratch) that will be used in subsequent labs. An Energy Monitor Logic function will be developed in Lab3.